Agentic架构 4：规划（Planning）

> **规划（Planning）** —— 与ReAct模型中逐步响应信息的方式不同，Planning Agent首先将一个**复杂任务分解**为一系列更小、更易管理的子目标。它在**采取任何行动之前**就创建了一个完整的"作战计划"。这种主动式方法为多步骤任务带来了**结构性、可预测性和效率**。

定义

规划架构涉及一个Agent，它在开始执行之前，明确地**将一个复杂目标分解为一系列详细的子任务**。这个初始规划阶段的输出是一个**具体的、逐步执行的计划**，然后Agent按部就班地遵循该计划来达成解决方案。

高层工作流

1. **接收目标：** Agent被赋予一个复杂任务。
2. **规划：** 一个专门的 **"规划器"组件** 分析目标，并生成一个实现该目标所需的**子任务有序列表**。例如：`["查找事实A", "查找事实B", "使用A和B计算C"]`。
3. **执行：** 一个"执行器"组件接收计划，并按**顺序执行**每个子任务，**根据需要调用工具**。
4. **综合：** 一旦计划中的所有步骤完成，最终组件将执行步骤的结果综合成一个**连贯的最终答案**。

适用场景 / 应用

- **多步骤工作流：** 适用于**操作顺序已知且至关重要**的任务，例如生成一份需要获取数据、处理数据然后进行总结的报告。
- **项目管理助手：** 将"发布一个新功能"这样的大目标分解为不同团队的子任务。
- **教育辅导：** 创建教学计划，引导学生从基础原理到高级应用，掌握特定概念。

优势与劣势

- **优势：**
  - **结构化且可追溯： 整个工作流预先规划**，使Agent的过程透明且易于调试。
  - **高效：** 对于可预测的任务，可能比ReAct更高效，因为它**避免了步骤间不必要的推理循环**。
- **劣势：**
  - **应对变化能力脆弱：** 如果在执行过程中环境发生意外变化，预先制定的计划可能会失败。它的**适应性**不如ReAct Agent，后者可以在每一步之后改变主意。

阶段 0：基础与环境搭建

从标准的环境搭建流程开始：安装库，并为 Nebius、LangSmith 以及 Tavily 网络搜索工具配置 API 密钥。

步骤 0.1：安装核心库

安装标准套件的库，包括更新的 `langchain-tavily` 包，以解决弃用警告问题。

```
pip install -q -U langchain-nebius langchain langgraph rich python-dotenv
langchain-tavily
```

步骤 0.2：导入库与配置密钥

导入必要的模块，并从 `.env` 文件中加载 API 密钥。

需要执行的操作： 在此目录下创建一个 `.env` 文件，并填入密钥：

```
NEBIUS_API_KEY="your_nebius_api_key_here"
LANGCHAIN_API_KEY="your_langsmith_api_key_here"
TAVILY_API_KEY="your_tavily_api_key_here"
```

```python
import os
import re
from typing import List, Annotated, TypedDict, Optional
from dotenv import load_dotenv

# LangChain 组件
from langchain_nebius import ChatNebius
from langchain_core.messages import BaseMessage, ToolMessage
from pydantic import BaseModel, Field
from langchain_core.tools import tool
from langchain_core.messages import SystemMessage
from langchain_tavily import TavilySearch

# LangGraph 组件
from langgraph.graph import StateGraph, END
from langgraph.graph.message import AnyMessage, add_messages
from langgraph.prebuilt import ToolNode, tools_condition

# 美化打印输出
from rich.console import Console
from rich.markdown import Markdown

# --- API 密钥与追踪配置 ---
load_dotenv()  # 加载 .env 文件中的环境变量

# 配置 LangSmith 追踪功能
os.environ["LANGCHAIN_TRACING_V2"] = "true"  # 启用 LangSmith 追踪 v2 版本
os.environ["LANGCHAIN_PROJECT"] = "Agentic Architecture - Planning
(Nebius)"  # 设置追踪项目名称

# 校验必需的环境变量是否已配置
for key in ["NEBIUS_API_KEY", "LANGCHAIN_API_KEY", "TAVILY_API_KEY"]:
    if not os.environ.get(key):
        print(f"{key} not found. Please create a .env file and set it.")

print("Environment variables loaded and tracing is set up.")
```

阶段 1：基线——反应式 Agent（ReAct）

为了体会规划的价值，首先需要一个基线。使用在上一笔记中构建的 ReAct Agent。这个 Agent 是智能的，但目光短浅——它是一步一步地摸索出前进路径的。

步骤 1.1：重建 ReAct Agent

快速重建 **ReAct Agent**。其核心特性是一个**循环**，在该循环中，Agent 的**输出在每次工具调用后都会被路由回自身**，使其能够根据最新信息重新评估并决定下一步行动。

```
# 初始化 Rich 控制台，用于美化打印输出
console = Console()

# 定义智能体状态结构，用于管理对话历史
class AgentState(TypedDict):
    """
    智能体状态结构，用于管理对话历史。
    使用 Annotated 类型与 add_messages 函数配合，实现消息列表的自动合并更新。
    当向状态中添加新消息时，LangGraph 会自动将新消息追加到现有消息列表末尾。
    """
    messages: Annotated[list[AnyMessage], add_messages]  # 对话消息列表，支持
自动追加更新

# 1. 定义来自 Tavily 包的基础搜索工具
tavily_search_tool = TavilySearch(max_results=2)  # 设置最大返回结果数为 2，保
持上下文简洁

# 2. 简化自定义工具
#    .invoke() 方法已经返回干净的字符串，因此直接传递即可
@tool
def web_search(query: str) -> str:
    """使用 Tavily 执行网络搜索，并将结果以字符串形式返回。"""
    console.print(f"--- TOOL: Searching for '{query}'...")  # 在控制台输出搜
索状态提示
    results = tavily_search_tool.invoke(query)  # 调用 Tavily 搜索工具
    return results  # 直接返回搜索结果字符串

# 3. 定义大语言模型并将其绑定到自定义工具
# 使用 Llama 3.1 8B 指令微调版本，temperature=0 使输出更具确定性
llm = ChatNebius(model="meta-llama/Meta-Llama-3.1-8B-Instruct",
temperature=0)

# 将 web_search 工具绑定到 LLM，使模型具备工具感知能力
llm_with_tools = llm.bind_tools([web_search])

# 4. 智能体节点，使用系统提示词强制每次只调用一个工具
def react_agent_node(state: AgentState):
    """
    ReAct 智能体节点，负责调用 LLM 进行推理和决策。
    通过系统提示词强制 LLM 每次只调用一个工具，避免多工具并发调用，确保智能体能够逐步推
理并适应工具返回的结果。
    """
```

```
    console.print("--- REACTIVE AGENT: Thinking... ---")  # 在控制台输出智能体思考状态提示

    # 构建包含系统提示词的消息列表，强制单次只调用一个工具
    messages_with_system_prompt = [
        SystemMessage(content="You are a helpful research assistant. You must call one and only one tool at a time. Do not call multiple tools in a single turn. After receiving the result from a tool, you will decide on the next step.")
    ] + state["messages"]

    # 调用已绑定工具的 LLM，获取模型响应
    response = llm_with_tools.invoke(messages_with_system_prompt)

    # 返回响应消息，LangGraph 会自动将其追加到状态的消息列表中
    return {"messages": [response]}

# 5. 在 ToolNode 中使用修复后的自定义工具
tool_node = ToolNode([web_search])  # 工具执行节点，负责执行 web_search 工具

# 构建具有 ReAct 特征循环的工作流图
react_graph_builder = StateGraph(AgentState)

# 向图中添加节点
react_graph_builder.add_node("agent", react_agent_node)  # ReAct 智能体节点
react_graph_builder.add_node("tools", tool_node)         # 工具执行节点

# 设置入口节点为智能体节点
react_graph_builder.set_entry_point("agent")

# 添加条件边：从智能体节点出发，根据 tools_condition 路由函数决定下一跳
# tools_condition 是 LangGraph 预置的条件路由函数，检查消息中是否包含 tool_calls
react_graph_builder.add_conditional_edges("agent", tools_condition)

# 添加从工具节点返回智能体节点的边，形成循环
# 这是 ReAct 模式的核心：支持多轮思考-行动迭代，实现多步推理
react_graph_builder.add_edge("tools", "agent")

# 编译图，生成可执行的应用实例
react_agent_app = react_graph_builder.compile()

print("Reactive (ReAct) agent compiled successfully.")
```

步骤 1.2：在计划导向型问题上测试反应式 Agent

给 ReAct Agent 一个任务，该任务需要两个**不同的数据收集步骤**，然后进行最终计算。这将测试其在没有预先计划的情况下管理多步骤工作流的能力。

```
# 定义以计划为中心的多步查询问题
# 该查询涉及多个步骤：
# 1. 查找法国、德国、意大利首都的人口
```

```
# 2. 计算三国首都人口总和
# 3. 与美国人口比较，判断哪个更大
plan_centric_query = """
Find the population of the capital cities of France, Germany, and Italy.
Then calculate their combined total.
Finally, compare that combined total to the population of the United
States, and say which is larger.
"""

# 在控制台以黄色粗体显示测试信息，表明正在测试反应式智能体在以计划为中心的查询上的表现
console.print(f"[bold yellow]Testing REACTIVE agent on a plan-centric
query:[/bold yellow] '{plan_centric_query}'\n")

# 初始化变量，用于存储最终输出状态
final_react_output = None

# 流式执行 ReAct 智能体工作流，以 "values" 模式逐个接收状态快照
# 这允许观察智能体在多轮思考-行动循环中的中间状态
for chunk in react_agent_app.stream({"messages": [("user",
plan_centric_query)]}, stream_mode="values"):
    # 每次迭代更新最终状态，循环结束后 final_react_output 为最后一步的状态
    final_react_output = chunk

    # 输出当前状态的标题（紫色粗体），便于观察执行过程中的每一步
    console.print(f"--- [bold purple]Current State Update[/bold purple] ---
")

    # 获取当前状态快照中的最后一条消息（即最新响应）
    # 使用 pretty_print() 方法以美观格式打印消息内容，便于调试和理解智能体行为
    chunk['messages'][-1].pretty_print()

    # 在每条消息后打印空行分隔，增强可读性
    console.print("\n")

# 输出最终结果的标题（红色粗体）
console.print("\n--- [bold red]Final Output from Reactive Agent[/bold red]
---")

# 获取最终状态中的最后一条消息内容（即智能体的最终回答）
# 使用 Markdown 格式渲染，支持富文本显示
console.print(Markdown(final_react_output['messages'][-1].content))
```

> **关于输出的讨论：** ReAct Agent 成功完成了任务。通过观察流式输出追溯其逐步推理过程：
>
> 1. 它首先决定搜索巴黎的人口。
> 2. 收到结果后，将其整合到记忆中，然后决定下一步是搜索柏林的人口。
> 3. 最后，收集到这两条信息后，它执行计算并提供最终答案。
>
> 虽然它成功了，但这种迭代发现过程并不总是最高效的。对于这样可预测的任务，**Agent每一步都需要调用LLM来生成下一步行动**。这为展示规划型 Agent 的价值奠定了基础。

阶段 2：进阶方法——规划型 Agent

构建一个 Agent 拥有一个专门的**规划器**来创建完整的任务列表，一个**执行器**来执行计划，以及一个**合成器**来整合最终结果。

步骤 2.1：定义规划器、执行器和合成器节点

为新的 Agent 创建核心组件：

1. `Planner`： 一个基于 **LLM** 的节点，**接收用户请求并输出结构化的计划**。
2. `Executor`： 一个节点，接收计划，使用工具执行**下一步**，并记录结果。
3. `Synthesizer`： 一个基于 **LLM** 的最终节点，**接收所有收集到的结果并生成最终答案**。

```
# Pydantic 模型，用于确保规划器的输出是结构化的步骤列表
class Plan(BaseModel):
    """用于回答用户查询所需执行的工具调用计划。"""
    steps: List[str] = Field(description="一个工具调用列表，按顺序执行后将回答用户查询。")

# 定义规划智能体的状态结构
class PlanningState(TypedDict):
    """
    规划智能体的状态结构。
    属性:user_request: 用户的原始请求
        plan: 待执行的步骤列表（规划器生成）
        intermediate_steps: 已执行步骤的结果列表
        final_answer: 综合所有步骤后生成的最终答案
    """
    user_request: str                       # 用户原始请求
    plan: Optional[List[str]]               # 规划器生成的步骤列表
    intermediate_steps: List[ToolMessage]   # 中间步骤的执行结果
    final_answer: Optional[str]             # 综合生成的最终答案

def planner_node(state: PlanningState):
    """生成用于回答用户请求的行动计划。"""
    console.print("--- PLANNER: Decomposing task... ---")  # 输出规划阶段提示

    # 将基础 LLM 包装为结构化输出模型，确保输出符合 Plan 的结构定义
    planner_llm = llm.with_structured_output(Plan)

    # 使用更明确的提示词，包含清晰的示例（Few-shot 学习）
    # 通过示例引导模型正确理解输出格式要求
    prompt = f"""You are an expert planner. Your job is to create a step-by-step plan to answer the user's request.
        Each step in the plan must be a single call to the `web_search` tool.

        **Instructions:**
        1. Analyze the user's request.
        2. Break it down into a sequence of simple, logical search queries.
        3. Format the output as a list of strings, where each string is a single valid tool call.
```

```
        **Example:**
        Request: "What is the capital of France and what is its
population?"
        Correct Plan Output:
        [
            "web_search('capital of France')",
            "web_search('population of Paris')"
        ]

        **User's Request:**
        {state['user_request']}
    """

    # 调用 LLM 生成计划
    plan_result = planner_llm.invoke(prompt)

    # 使用 plan_result.steps 而非 plan.steps，避免与变量名 'plan' 混淆
    console.print(f"--- PLANNER: Generated Plan: {plan_result.steps} ---")
    return {"plan": plan_result.steps}

def executor_node(state: PlanningState):
    """执行计划中的下一步骤。"""
    console.print("--- EXECUTOR: Running next step... ---")  # 输出执行阶段提
示

    # 获取当前计划并取出第一个步骤
    plan = state["plan"]
    next_step = plan[0]

    # 使用稳健的正则表达式解析工具调用，同时支持单引号和双引号
    # 正则表达式解析格式：tool_name('query') 或 tool_name("query")
    match = re.search(r"(\w+)\((?:\"|\')(.*?)(?:\"|\')\)", next_step)
    if not match:
        # 如果解析失败，默认使用 web_search 工具，整个字符串作为查询
        tool_name = "web_search"
        query = next_step
    else:
        # 成功解析，提取工具名称和查询参数
        tool_name, query = match.groups()[0], match.groups()[1]

    console.print(f"--- EXECUTOR: Calling tool '{tool_name}' with query
'{query}' ---")

    # 调用 Tavily 搜索工具执行查询
    result = tavily_search_tool.invoke(query)

    # 创建 ToolMessage 记录工具执行结果
    # tool_call_id 使用查询的哈希值生成，确保唯一性
    tool_message = ToolMessage(
        content=str(result),
        name=tool_name,
        tool_call_id=f"manual-{hash(query)}"
    )
```

```
    return {
        "plan": plan[1:],  # 从计划中移除已执行的步骤
        "intermediate_steps": state["intermediate_steps"] + [tool_message]
# 追加执行结果
    }

def synthesizer_node(state: PlanningState):
    """综合中间步骤的结果，生成最终答案。"""
    console.print("--- SYNTHESIZER: Generating final answer... ---")  # 输
出综合阶段提示

    # 构建上下文信息，将所有工具返回的结果拼接成字符串
    context = "\n".join([f"Tool {msg.name} returned: {msg.content}" for msg
in state["intermediate_steps"]])

    # 构建综合提示词，要求模型基于用户请求和收集的数据生成最终答案
    prompt = f"""You are an expert synthesizer. Based on the user's request
and the collected data, provide a comprehensive final answer.

    Request: {state['user_request']}
    Collected Data:
    {context}
    """

    # 调用 LLM 生成最终答案
    final_answer = llm.invoke(prompt).content
    return {"final_answer": final_answer}

print("Planner, Executor, and Synthesizer nodes defined.")
```

步骤 2.2：构建规划型 Agent 图结构

把新节点组装成一个图结构。流程将是：`Planner` → `Executor`（循环） → `Synthesizer`。

```
def planning_router(state: PlanningState):
    """
    规划智能体的路由函数，根据当前计划状态决定下一步执行路径。
    路由逻辑：
    - 如果计划列表为空，表示所有步骤已执行完毕，路由到综合节点生成最终答案
    - 如果计划列表非空，表示还有未执行的步骤，路由到执行节点继续执行
    """
    # 检查计划列表中是否还有待执行的步骤
    if not state["plan"]:
        # 计划已执行完毕，路由到综合节点
        console.print("--- ROUTER: Plan complete. Moving to synthesizer. --
-")
        return "synthesize"
    else:
        # 计划中还有步骤，路由到执行节点继续执行
        console.print("--- ROUTER: Plan has more steps. Continuing
```

```
execution. ---")
        return "execute"

# 创建规划智能体的状态图构建器
planning_graph_builder = StateGraph(PlanningState)

# 向图中添加三个核心节点
planning_graph_builder.add_node("plan", planner_node)            # 规划节
点：生成执行计划
planning_graph_builder.add_node("execute", executor_node)        # 执行节
点：执行计划中的步骤
planning_graph_builder.add_node("synthesize", synthesizer_node)  # 综合节
点：生成最终答案

# 设置入口节点为规划节点
planning_graph_builder.set_entry_point("plan")

# 添加条件边：从规划节点出发，根据路由函数决定下一跳
# 规划节点执行后，要么进入执行节点（有步骤），要么进入综合节点（无步骤）
planning_graph_builder.add_conditional_edges(
    "plan",
    planning_router,  # 路由函数
    {"execute": "execute", "synthesize": "synthesize"}  # 路由映射表
)

# 添加条件边：从执行节点出发，同样根据路由函数决定下一跳
# 执行节点执行后，要么继续执行下一个步骤（循环），要么进入综合节点（计划完成）
planning_graph_builder.add_conditional_edges(
    "execute",
    planning_router,  # 路由函数
    {"execute": "execute", "synthesize": "synthesize"}  # 路由映射表
)

# 添加从综合节点到结束节点的边，流程结束
planning_graph_builder.add_edge("synthesize", END)
# 编译图，生成可执行的应用实例
planning_agent_app = planning_graph_builder.compile()
print("Planning agent compiled successfully.")
```

阶段 3：正面比较

在新的规划型 Agent 上运行相同的任务，并将其执行流程和最终输出与反应式 Agent 进行比较。

```
# 在控制台以绿色粗体显示测试信息，表明正在测试规划智能体在以计划为中心的查询上的表现
# 这与之前测试反应式智能体使用相同的查询，便于对比两种架构的性能差异
console.print(f"[bold green]Testing PLANNING agent on the same plan-centric
query:[/bold green] '{plan_centric_query}'\n")

# 正确初始化规划智能体的状态
# 注意：intermediate_steps 必须初始化为空列表，用于存储中间步骤的执行结果
```

```
# 规划节点将生成 plan 列表，执行节点会逐步消费 plan 并将结果追加到
intermediate_steps
initial_planning_input = {
    "user_request": plan_centric_query,   # 用户请求
    "intermediate_steps": []              # 初始化为空列表，用于收集工具执行结果
}

# 调用规划智能体应用，执行完整工作流
# 与流式执行不同，invoke() 会等待整个流程完成后返回最终状态
final_planning_output = planning_agent_app.invoke(initial_planning_input)

# 输出最终结果的标题（绿色粗体）
console.print("\n--- [bold green]Final Output from Planning Agent[/bold
green] ---")

# 获取最终状态中的 final_answer 字段（综合节点生成的最终答案）
# 使用 Markdown 格式渲染，支持富文本显示
console.print(Markdown(final_planning_output['final_answer']))
```

> **关于输出的讨论：** 流程上的差异一目了然。 第一步就是 `Planner` 创建了一个完整的、明确的计划：`['web_search("population of Paris")', 'web_search("population of Berlin")']`。 然后 Agent 有条不紊地执行了这个计划，**无需在步骤之间停下来思考**。这个过程：
>
> - **更透明：** 我们可以在 Agent 开始之前就看到它的完整策略。
> - **更稳健：** 因为它在遵循一套清晰的指令，所以不太容易偏离方向。
> - **可能更高效：** 它避免了步骤之间进行推理所需的额外 LLM 调用。
>
> 这展示了**规划**在那些步骤可以**预先确定的任务**中的强大能力。

阶段 4：定量评估

为了规范化比较，使用 LLM 作为评判者对两种 Agent 进行评分，重点关注它们解决问题过程的**质量和效率**。

```
class ProcessEvaluation(BaseModel):
    """用于评估智能体问题解决过程的结构。"""
    task_completion_score: int = Field(description="任务完成度评分，1-10分（智
能体是否成功完成任务）。")
    process_efficiency_score: int = Field(description="过程效率评分，1-10分
（智能体过程的效率和直接性，分数越高表示路径越符合逻辑、绕路越少）。")
    justification: str = Field(description="各项评分的简要说明。")

# 将基础 LLM 包装为结构化输出模型，确保输出符合 ProcessEvaluation 的结构定义
judge_llm = llm.with_structured_output(ProcessEvaluation)

def evaluate_agent_process(query: str, final_state: dict):
    """
    评估智能体解决问题过程的逻辑性和效率。
    参数:query: 用户原始查询
        final_state: 智能体执行完成后的最终状态
    返回:ProcessEvaluation: 包含评分和说明的评估结果
```

```
    """
    # 根据智能体类型构建不同的轨迹字符串
    # ReAct 智能体：消息历史存储在 'messages' 字段中
    if 'messages' in final_state:
        # 提取每条消息的类型和内容，用换行符连接形成完整对话轨迹
        trace = "\n".join([f"{m.type}: {str(m.content)}" for m in
final_state['messages']])
    else:
        # 规划智能体：计划存储在 'plan' 字段，执行结果存储在 'intermediate_steps'
字段
        trace = f"Plan: {final_state.get('plan', [])}\nSteps:
{final_state.get('intermediate_steps', [])}"

    # 构建评估提示词，要求模型扮演 AI 智能体评审专家角色
    # 评估重点：过程的逻辑性和效率
    prompt = f"""You are an expert judge of AI agents. Evaluate the agent's
process for solving the task on a scale of 1-10.
    Focus on whether the process was logical and efficient.

    **User's Task:** {query}
    **Full Agent Trace:**\n```\n{trace}\n```
    """
    # 调用 LLM 生成评估结果
    return judge_llm.invoke(prompt)

# 输出反应式智能体过程评估的标题
console.print("--- Evaluating Reactive Agent's Process ---")

# 对 ReAct 智能体的执行过程进行评估
react_agent_evaluation = evaluate_agent_process(plan_centric_query,
final_react_output)

# 使用 model_dump() 将 Pydantic 模型序列化为字典格式并打印
console.print(react_agent_evaluation.model_dump())

# 输出规划智能体过程评估的标题
console.print("\n--- Evaluating Planning Agent's Process ---")

# 对规划智能体的执行过程进行评估
planning_agent_evaluation = evaluate_agent_process(plan_centric_query,
final_planning_output)

# 使用 model_dump() 将 Pydantic 模型序列化为字典格式并打印
console.print(planning_agent_evaluation.model_dump())
```

> **关于输出的讨论：** 评判者的评分量化了两种方法之间的差异。 两个 Agent 可能都获得了较高的 `task_completion_score`（**任务完成度评分**），因为它们最终都找到了答案。 然而，**规划型 Agent** 将获得显著更高的 `process_efficiency_score`（**过程效率评分**）。评判者的理由将强调，与 ReAct Agent 逐步探索的过程相比，**其预先制定的计划是解决问题更直接、更合乎逻辑的方式**。
>
> 这种评估证实了我们的假设：对于**解决方案路径可预测**的问题，**规划架构**提供了一种**更结构化、更透明且更高效**的方法。

结论

在本笔记中，我们实现了**规划**架构，并将其与 **ReAct** 模式进行了直接对比。通过强制 Agent 在执行前首先构建一个全面的计划，我们在处理**定义明确的多步骤任务**时，在透明度、稳健性和效率方面获得了显著优势。

虽然 ReAct 在下一步未知的探索性场景中表现出色，但规划在**可以预先规划解决方案路径**的场景中更显优势。理解这种权衡对于系统设计者来说至关重要。为正确的问题选择正确的架构，是构建有效且智能的 AI Agent 的关键技能。规划模式是该工具包中的一项重要工具，它为**复杂的、可预测的工作流**提供了所需的结构。